

Distance Comparison Operations Are Not Silver Bullets in Vector Similarity Search: An Experimental Study on Their Merits and Limits

Zhuanglin Zheng, Yuxiang Zeng, Chenchen Liu, Yunzhen Chi, Binhan Yang, Yongxin Tong
SKLCCSE Lab, BDBC and IRI, Beihang University, Beijing, China
{zzlin, yxzeng, 23371020, chiyz, yangbh, yxtong}@buaa.edu.cn

Abstract—Distance Comparison Operations (DCOs), which determine whether the distance between a data vector and a query is within a given threshold, are a critical performance bottleneck in vector similarity search. Recent DCO methods that avoid full-dimensional distance computations promise significant speedups, but their readiness for production vector database systems remains an open question. To address this, we conduct a comprehensive benchmark of 8 DCO algorithms across 10 datasets (with up to 100M vectors and 12,288 dimensions) and diverse hardware configurations (CPUs with/without SIMD, and GPUs). Our study reveals that *these methods are not silver bullets*: their efficiency is highly sensitive to data dimensionality, degrades under out-of-distribution queries, and is unstable across hardware. Yet, our evaluation also demonstrates often-overlooked merits: they can accelerate index construction and data updates. Despite these benefits, their unstable performance, which can be slower than a full-dimensional scan, leads us to conclude that *no current DCO method is ready for deployment*.

Index Terms—Vector Database, Similarity Search, Benchmark.

I. INTRODUCTION

Vector similarity search is a fundamental component of modern data-intensive applications, including recommendation systems, search engines, and Retrieval-Augmented Generation (RAG). A central challenge in this domain is improving search efficiency. While the majority of research has focused on designing index structures, a recent and promising line of work optimizes efficiency at a more granular operator level.

These studies [1]–[5] identify the *Distance Comparison Operation (DCO)* (which determines whether the distance between a data vector and a query vector is within a given threshold) as a critical efficiency bottleneck. Traditional vector database systems often perform DCOs by exhaustively computing the full-dimensional distance, a method denoted as Full Dimension Scanning (FDScanning). In contrast, state-of-the-art methods [1]–[4] leverage techniques like hypothesis testing or prediction to scan only partial dimensions and infer the comparison result. These approaches have been reported to improve query throughput by 2–4 $\times$ over FDScanning with almost no loss in recall.

The impressive results *raise* a natural yet critical question: *Are current DCO methods ready for deployment in production vector databases?* To answer this, we analyzed the evaluation protocols of prior research. As summarized in Table I, we identified several key limitations in their evaluations:

- **Narrow Dimensionality Coverage:** They are largely confined to the dimensionality range of 128–960, neglecting lower-dimensional and, particularly, ultra-high-dimensional data common in embeddings from LLMs.
- **Insufficient Method Comparison:** They lack comprehensive comparisons among state-of-the-art methods.
- **Restricted Hardware Configuration:** They typically use CPUs with SIMD disabled, which does not reflect the computing capabilities of a modern server.
- **Limited Application Scenario:** They mainly focus on in-distribution queries, overlooking practical scenarios like Out-of-Distribution (OOD) queries and index building.

These limitations prevent a definitive answer to the previous work and motivate our work. To this end, we conduct a comprehensive benchmark of 8 DCO methods across 10 datasets and different hardware conditions. Our investigation reveals that *these methods are not silver bullets*. Their effectiveness is highly contingent, and their adoption requires careful consideration. Specifically, we have the following major findings:

(1) **Dimensionality Sensitivity:** The performance of DCOs is highly sensitive to data dimensionality. Most methods excel between 128–1536 dimensionality, but their advantage can degrade significantly outside this range, often falling behind the naive method FDScanning by up to 0.3–0.4 $\times$.

(2) **Robustness Dilemma:** Existing methods face a dual challenge. On one hand, they are vulnerable to OOD queries that are common in applications like multi-modal retrieval. On the other hand, their efficiency gains are unstable across hardware. For instance, enabling SIMD can reduce or even reverse their advantage over FDScanning.

(3) **Benefit Beyond Query:** An often-overlooked merit of DCO methods lies in accelerating index construction and data updates by up to 64% and 63%, respectively.

(4) **No Universal Winner:** There is no single dominant method. Their performance ranking varies drastically with data characteristics, query distribution, and hardware.

Contribution. In summary, this paper makes the following major contributions:

- We present the first comprehensive benchmark for evaluating DCOs in vector similarity search.
- Through extensive evaluation, we provide a detailed analysis of the merits and limitations of existing DCO methods. The in-depth analysis leads to the final answer:

TABLE I: Comparisons of evaluation setup in prior work on Distance Comparison Operation (DCO)

Related Work	#(Dataset)	Dimensionality	Scalability	#(Algorithm) Tested	Compared Work	Hardware Environment	Application Scenario	Distance Metric*	OOD Query*
[1]	6	256 ~ 960	5 million	3	N/A	CPU w/o SIMD	Query	Euclidean	×
[2]	6	256 ~ 960	5 million	3	[1]	CPU w/o SIMD	Query	Euclidean	×
[3]	8	128 ~ 960	100 million	5	[1]	CPU w/o SIMD	Query	Euclidean, IP	✓
[4]	6	96 ~ 4096	1 million	3	[1]	CPU ± SIMD	Query	Euclidean	×
Ours	10	96 ~ 12288	100 million	8	[1]–[4]	CPU ± SIMD & GPU	Query, Update, Index Construction	Euclidean, IP	✓

* IP: Inner Product; OOD: Out-of-Distribution, Ref. [3] uses only synthetic data (query vectors) for OOD evaluation.

TABLE II: Summary of frequently used notations

Notation	Description
$\mathcal{O}$	A data $\mathcal{O}$ of N vectors, each of dimensionality D
$\mathcal{S}$	Query answer
q, k	Query vector q and the number k of nearest neighbors
$dis(o_1, o_2)$	Distance between two vectors o_1 and o_2
τ	Distance threshold
d	Number of scanned dimensions during performing DCO

the current DCO methods, while promising, are not yet ready for production deployment.

- We derive practical guidelines for selecting the optimal DCO method under different scenarios and highlight future research directions. To foster further work, we have open-sourced the benchmark on GitHub [6].

Road Map. The rest of this paper is structured as follows. Sec. II defines the DCO problem, and Sec. III introduces existing DCO algorithms. Next, Sec. IV and Sec. V detail the benchmark setup and analyze the evaluation results. Finally, Sec. VI reviews related work and Sec. VII concludes the paper.

II. PRELIMINARY

In this section, we establish the basic concepts and formally define the *Distance Comparison Operation (DCO)*. A summary of frequently used notations is provided in Table II.

A. Basic Concepts

Before defining the DCO, we introduce two basic concepts: vector data and vector similarity search.

Definition 1 (Vector Data): A vector data (“vector” as short) o is defined as an ordered sequence $(x_1, x_2, \dots, x_D)$. Here, D is the vector’s dimension, and x_i represents the i -th coordinate. The function $dis(o_1, o_2)$ denotes the distance between two vectors o_1 and o_2 , which is a measure of their similarity.

We use $\mathcal{O}$ to denote a dataset containing N vectors, each with D dimensions. In practice, vector datasets can be classified into three kinds based on their dimensionality [7], [8]:

- **Low-Dimensional Vector Dataset:** These are defined by a moderate dimensionality ($D \in [10, 100]$). They are prevalent in classical statistics and traditional machine learning (ML) applications [9], [10].
- **High-Dimensional Vector Dataset:** These datasets exhibit a large number of dimensions ($D \in (100, 1000]$). They usually consist of embedding vectors generated by

deep learning models in domains like computer vision and natural language processing [11], [12].

- **Ultra-High-Dimensional Vector Dataset:** This kind encompasses datasets with dimensionality on the order of 10^3 or higher. They often arise from raw sensor data and embedding layers of LLMs [13], [14].

Distance functions are also diverse but can be broadly categorized into metric and non-metric distances. Commonly used vector distance functions include Euclidean distance, inner product, and cosine similarity. For simplicity, we use Euclidean distance as the representative distance function for defining the problem and describing the algorithms.

Definition 2 (Vector Similarity Search): Given a vector dataset $\mathcal{O}$, a query vector $q \in \mathbb{R}^D$, and a positive integer k , vector similarity search aims to find the k nearest neighbor (KNN) vectors $\mathcal{S} \subseteq \mathcal{O}$ to the query vector q satisfying

$$|\mathcal{S}| = k \text{ and } \forall v \in \mathcal{S}, \forall u \in (\mathcal{O} \setminus \mathcal{S}), dis(v, q) \leq dis(u, q)$$

Remark. The scalability of exact solutions to vector similarity search is severely limited by the curse of dimensionality [15]. This has led to a rich line of research focused on *approximate algorithms* that trade exactness for efficiency, with the primary goal of *maximizing the recall*. The recall of the query answer $\mathcal{S}$ against the ground truth $\mathcal{S}^*$ is defined as:

$$\text{recall} = \frac{|\mathcal{S} \cap \mathcal{S}^*|}{k} \quad (1)$$

B. Definition of Distance Comparison Operation

Overview of Vector Distance Operations. Prior research in vector similarity search relies heavily on two types of vector distance operations: *distance computation operations* and *distance comparison operations*.

- **Distance Computation Operation:** This operation calculates the *exact distance* between two vectors. It is computationally intensive but can often be accelerated by hardware like SIMD instructions and GPUs.
- **Distance Comparison Operation (DCO):** This operation determines whether the distance between two vectors exceeds a given threshold. Its efficiency enhancement stems from a key insight: computing exact distance is often unnecessary, since the comparison result can be derived by scanning only a subset of the dimensions. We formally define this operation below.

Definition 3 (Distance Comparison Operation (DCO) [1]): Given two vectors o, q and a distance threshold τ , the distance

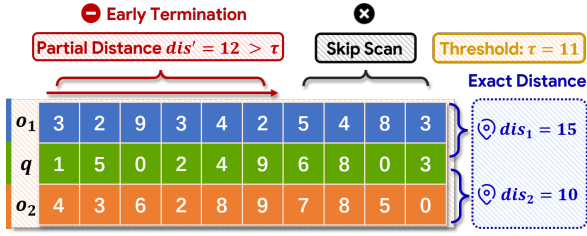


Fig. 1: Instances of Distance Comparison Operation (DCO)

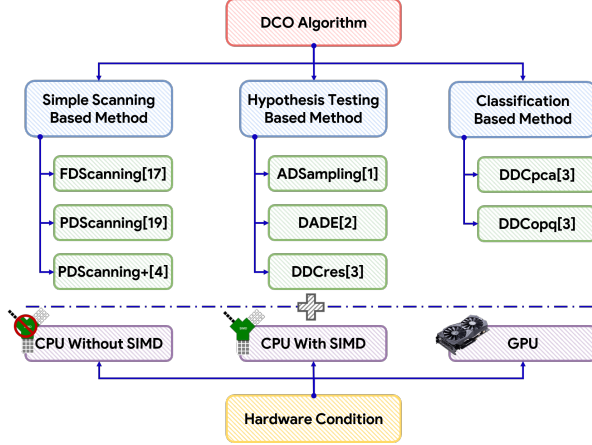


Fig. 2: Taxonomy of existing DCO methods

comparison operation (DCO) determines the true value of the statement $dis(o, q) \leq \tau$. If true, it also returns the exact distance. Otherwise, it only returns the negative result.

The distance threshold τ is typically the distance between the query vector and the current candidate for KNNs. The exact distance is returned in a positive case for two primary reasons: (1) such a case often requires scanning all dimensions, making the exact distance readily available; and (2) the vector o is closer than the current candidate, so its exact distance is required for ranking the final KNNs. The following example illustrates this operation.

Example 1: Fig. 1 shows two instances of DCO with data vectors o_1 and o_2 , a query vector q , and a distance threshold $\tau = 11$. A naive solution is to scan all dimensions, compute the Euclidean distances $dis(o_1, q) = 15$ and $dis(o_2, q) = 10$, and then compare them with τ . However, a full-dimensional scan is not always necessary. For o_1 , after scanning just the first 6 coordinates, the partial distance reaches 12, which already exceeds τ . This eliminates the need to scan the remaining dimensions of o_1 and thereby reduces computational cost.

III. EXISTING METHODS

In this section, we first introduce a taxonomy of existing DCO methods, and categorize them as *simple scanning based*, *hypothesis testing based* and *classification based* methods. We then review each category and finally present a comparative analysis of these methods.

Algorithm 1: Simple Scanning Based Framework

Input: Two vectors o, q and a distance threshold τ
Output: Whether $dis(o, q) \leq \tau$

- 1 Initialize the number of currently scanned dimensions $d \leftarrow 0$, partial distance $dis' \leftarrow 0$;
- 2 **while** the number of scanned dimensions $d < D$ **do**
- 3 Update partial distance dis' and increase d ;
- 4 **if** $dis' > \tau$ **then** // **only for PDS Scanning**
- 5 **return False**;
- 6 **return True**;

A. Method Taxonomy

The design of high-performance DCOs follows two paths: *dedicated hardware* and *optimized algorithms*. Hardware-based approaches usually leverage GPUs [16] for massive thread-level parallelism or SIMD instructions [17] for vector-level parallelism. However, as recent research focuses on algorithmic optimization strategies [1]–[3], [18], these hardware-centric methods are not our primary concern.

As shown in Fig. 2, we categorize these optimization algorithms into three kinds based on their core ideas: *simple scanning based*, *hypothesis testing based*, and *classification based*. These methods are complementary to hardware accelerators like GPUs and SIMD instructions, which will be evaluated later. The subsequent subsections will delve into each of these categories in detail, while Fig. 3 provides a high-level overview of their respective ideas and workflows.

B. Simple Scanning Based Methods

Simple scanning based methods solve the DCO problem by scanning dimensions of vectors o and q , through two strategies: *Full Dimension Scanning (FDS Scanning)* and *Partial Dimension Scanning (PDS Scanning)*.

- FDS Scanning [17] scans all dimensions to compute the exact distance and then compares it to the threshold τ .
- PDS Scanning [19] scans dimensions incrementally while maintaining a running partial distance. It terminates the scan as soon as the intermediate result proves that $dis(o, q) > \tau$, avoiding unnecessary computation.

General Framework. Alg. 1 presents the general framework of these DCOs. In lines 2-5, the framework scans both vectors' dimensions from scratch and maintains the partial distance dis' . A key distinction emerges in the **termination** condition: PDS Scanning immediately exits the loop when dis' exceeds the threshold τ , whereas FDS Scanning continues through all dimensions regardless of the intermediate results.

Optimization. The efficiency of this framework can be further enhanced by two techniques: Single Instruction Multiple Data (SIMD) and Principal Component Analysis (PCA).

- **Optimization via SIMD:** The partial distance computation can be parallelized at the instruction level using

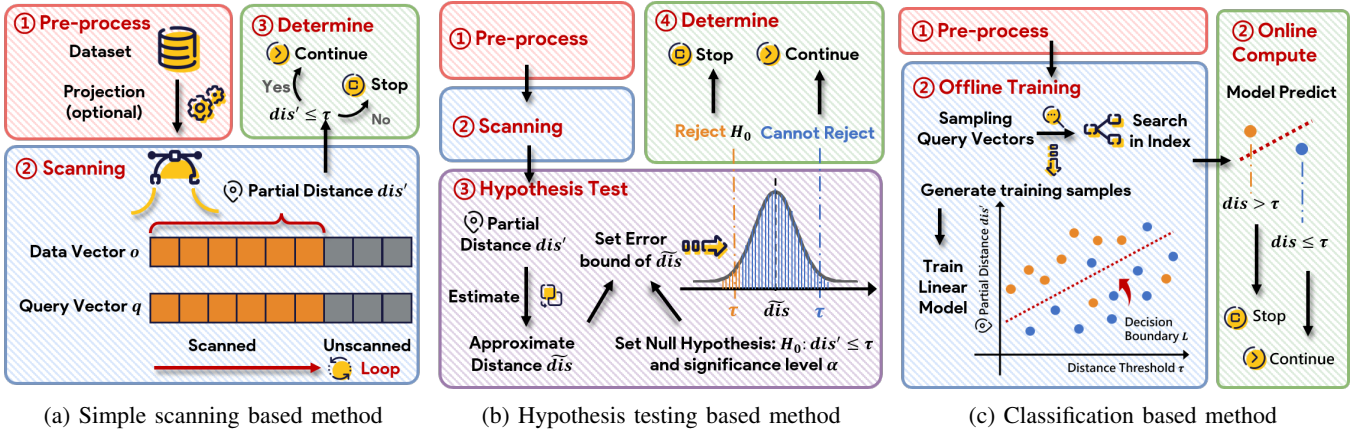


Fig. 3: The three categories of DCO methods: core ideas and main workflows

SIMD. For example, both FDScanning and PDScanning can partition the dimensions of a vector into contiguous blocks that align with the SIMD register width (e.g., 128-bit or wider). This enables the CPU to simultaneously perform arithmetic operations across multiple dimensions within a single instruction cycle. In this way, more than 4 dimensions can be processed together in a batch, which saves computational cost.

- **Optimization via PCA:** PCA transforms the original vector data into a new coordinate system, where dimensions are ordered by their variance contributions. This ensures that the preceding dimensions contribute more significantly to the overall (Euclidean) distance between vectors. PDScanning can leverage this property to enable early termination in lines 4-5 of Alg. 1. Despite its simplicity, this optimization method has not been empirically evaluated in prior work. We name it PDScanning+ [4] and conduct comprehensive experimental comparisons with other baselines in Sec. V.

C. Hypothesis Testing Based Methods

The anticipated efficiency gain of PDScanning over FDScanning stems from scanning fewer dimensions. This intuition motivates a class of approximate algorithms that estimate the full distance (denoted by $\tilde{dis}$) from a partial distance (dis') to traverse even fewer dimensions. Using the estimated distance within a probability inequality, these algorithms then perform a hypothesis test to accept or reject $dis(o, q) \leq \tau$. They can be classified into two categories based on the estimation strategies: (1) *residual distance estimator* and (2) *cross-term estimator*.

Strategy I. Residual Distance Estimator. This class of algorithms estimates the residual distance (i.e., the distance contributed by unscanned dimensions) based on the partial distance dis' from the currently scanned dimensions. There are two general methods for such estimators.

(1) Assuming Equal Contribution per Dimension. Under this assumption, the residual distance is estimated as $\sqrt{\frac{D-d}{d}}dis'$, leading to a full distance estimate $\sqrt{\frac{D}{d}}dis'$. To

enforce this assumption in practice, ADSampling [1] employs a random projection matrix $P \in \mathbb{R}^{d \times D}$ to map original vectors into a new coordinate system. This work also proves an error bound on the estimated distance as follows:

$$\mathbb{P} \left\{ \left| \sqrt{\frac{D}{d}}dis(Po, Pq) - dis(o, q) \right| \leq \epsilon dis(o, q) \right\} \geq 1 - 2e^{-c \cdot d \cdot \epsilon^2} \quad (2)$$

In this inequality, $\sqrt{\frac{D}{d}}dis(Po, Pq)$ is the estimated distance $\tilde{dis}$, c is a constant factor, and ϵ is the error tolerance. Thus, the inequality implies that $\tilde{dis} \leq (1 + \epsilon) \cdot dis$ holds with high probability $1 - 2e^{-cde^2}$. Moreover, this estimator ensures that $\tilde{dis} = dis$ when all original dimensions are preserved in the projection (i.e., $d = D$).

(2) Assuming Differing Contribution per Dimension. In contrast, DADE [2] is designed for scenarios where dimensions contribute differently to the distance, and treats dimensions with different weights. To achieve this goal, it performs a PCA over the vector dataset to obtain the **loading matrix** (denoted by $W \in \mathbb{R}^{D \times D}$) as the projection matrix. This ensures that the initial dimensions capture the largest possible contribution to the distance. Consequently, the result of DCOs can be quickly decided after scanning only a small subset of the leading dimensions.

Similarly, Deng *et al.* [2] also propose an error bound on the estimated distance as follows:

$$\mathbb{P} \left\{ \sqrt{\frac{\sum_{k=1}^D \lambda_k}{\sum_{k=1}^d \lambda_k}}dis(W_d^T o, W_d^T q) > (1 + \epsilon_d)dis(o, q) \right\} < \alpha \quad (3)$$

where λ_k is the k -th largest eigenvalue of the vector dataset's covariance matrix, α is the significance level, and ϵ_d is the **error tolerance** after scanning d dimensions. Both α and ϵ_d are user-defined parameters whose values are set empirically.

Strategy II. Cross-Term Estimator. This strategy employs an algebraic decomposition of the squared Euclidean distance into a sum of squared terms and cross-terms:

$$dis(o, q) = \|o\|^2 + \|q\|^2 - 2\langle o, q \rangle \quad (4)$$

$$= \|o\|^2 + \|q\|^2 - 2\langle od, qd \rangle - 2\langle or, qr \rangle \quad (5)$$

Algorithm 2: Hypothesis Testing Based Framework

Input: Two vectors o, q and a distance threshold τ

Output: Whether $\text{dis}(o, q) \leq \tau$

```
1 Set the null hypothesis  $H_0 : \text{dis} \leq \tau$  and its alternative
  hypothesis  $H_1 : \text{dis} > \tau$ ;
2 while the number of scanned dimensions  $d < D$  do
3   Update partial distance  $\text{dis}'$  and increase  $d$ ;
4   Set significance level  $\alpha$  by their parameter settings;
5   Compute a relaxed distance threshold  $(1 + \epsilon) \cdot \tau$ ;
    //  $\epsilon = \epsilon_d$  for DADE,  $\epsilon = 0$  for DDCres
6   Compute the estimated distance  $\widetilde{\text{dis}}$  based on  $\text{dis}'$ ;
7   if  $\widetilde{\text{dis}} > (1 + \epsilon) \cdot \tau$  then
8     Reject  $H_0$  with confidence and return False;
9 return True;
```

where $\|\cdot\|$ is the L_2 -norm, and $\langle \cdot, \cdot \rangle$ is the inner product. The sub-vectors od, qd and or, qr contain the scanned and unscanned dimensions, respectively. In Eq. (5), both $\|o\|$ and $\|q\|$ can be pre-processed, and $\langle od, qd \rangle$ can be computed directly from the scanned dimensions. Their collective partial result is denoted by dis' . This method thus focuses on estimating the remaining cross-term, $\langle or, qr \rangle$.

Yang *et al.* [3] propose a new method called DDCres to bound this term. DDCres [3] first centers the data vectors to have zero mean. It then assumes that both query and data vectors follow a Gaussian distribution, where σ_i^2 denotes the variance of the i -th dimension. Then, the expectation and variance of the remaining cross-term are derived as:

$$\mathbb{E}[\langle o_r, q_r \rangle] = \sum_{i=d+1}^D \mathbb{E}[o_i \cdot q_i] = \sum_{i=d+1}^D (\mathbb{E}[o_i] \cdot \mathbb{E}[q_i]) = 0 \quad (6)$$

$$\text{Var}[\langle o_r, q_r \rangle] = \sum_{i=d+1}^D (q_i \cdot \sigma_i)^2 \quad (7)$$

Consequently, the exact distance dis lies within a confidence interval $\text{dis}' \pm m \cdot 2\sqrt{\text{Var}[\langle o_r, q_r \rangle]}$ with high probability, where m is a deviation multiplier. To test if $\text{dis} > \tau$, the lower bound of this interval is used as the estimated distance $\widetilde{\text{dis}}$, i.e.,

$$\widetilde{\text{dis}} = \text{dis}' - m \cdot 2\sqrt{\text{Var}[\langle o_r, q_r \rangle]} \quad (8)$$

General Framework. The workflow of hypothesis testing based methods is shown in Alg. 2. Compared to simple scanning based methods, hypothesis testing based methods include an additional step in lines 5-7, which utilizes hypothesis testing to determine whether the null hypothesis holds. Correspondingly, the condition in line 8 is replaced by a check on whether the estimated distance $\widetilde{\text{dis}}$ exceeds a statistically derived error bound. If it does, the null hypothesis is rejected and the loop terminates early.

Beyond Euclidean Distance. The hypothesis testing based methods also support inner product and cosine similarity by transforming them into Euclidean distance. This requires the

Algorithm 3: Classification Based Method DDCpca

Input: Two vectors o, q and a distance threshold τ

Output: Whether $\text{dis}(o, q) \leq \tau$

```
1 // Training Phase (Offline)
2 Sample query vectors from dataset and parameters  $k$ ;
3 Generate training samples through vector similarity
  search for sampled queries using a specific index;
4 Train linear models  $M_{k,d}$ , denoting the model for
  query parameter  $k$  after  $d$  dimensions are scanned.;
5 // Computation Phase (Online)
6 while the number of scanned dimensions  $d < D$  do
7   Update the partial distance  $\text{dis}'$  and increase  $d$ ;
8   Use model  $M_{k,d}$  to predict for  $\text{dis}'$  and  $\tau$ ;
9   if this model predicts  $\text{dis}(o, q) > \tau$  then
10     return False;
11 return True;
```

vector dataset to be normalized first. Then, the inner product $\langle o, q \rangle$ is derived from the Euclidean distance $\text{dis}(o, q)$ as:

$$\langle o, q \rangle = 1 - 0.5 \cdot (\text{dis}(o, q))^2 \quad (9)$$

Since cosine similarity is equivalent to the inner product for normalized vectors, this extension applies to both metrics.

D. Classification Based Method

Main Idea. Other research [3] formulates the DCO problem as a binary classification task. Given the partial distance dis' and threshold τ , the goal of this task is to predict whether the full distance satisfies $\text{dis} \leq \tau$. These methods typically assume the query workload is known a priori (e.g., often modeled using the vector dataset's distribution). Under this assumption, they utilize a fixed vector index and sampled queries (including both integers k and query vectors q) to generate sufficient training samples. Subsequently, a distinct linear model $M_{k,d}$ is trained for each combination of the integer k and the number d of scanned dimensions.

Representative Algorithm. Alg. 3 illustrates a representative algorithm DDCpca to this kind. The offline phase generates training samples by performing similarity searches with varying k on a fixed vector index. These samples are used to train a set of linear models $M_{k,d}$, each corresponding to a query parameter k after d dimensions are scanned. During the online computation phase, DDCpca feeds the partial distance dis' and distance threshold τ into the model $M_{k,d}$. The process terminates early if the model predicts $\text{dis}(o, q) > \tau$.

Variant. Yang *et al.* [3] propose another classification based method, named DDCopq. Unlike DDCpca, DDCopq trains a single linear model M_k for each query parameter k , using approximate distances derived from Product Quantization (PQ). During a DCO, this model first predicts if $\text{dis}(o, q) > \tau$. If the result is negative, it performs a distance computation operation by scanning all dimensions to verify the result.

TABLE III: Comparisons of existing methods for Distance Comparison Operation (DCO)

Category	Methods	Exactness	Assumption ¹	Time Complexity Breakdown ²		
				Offline Pre-processing	Online Pre-processing	Online Computation
Simple Scanning	FDScanning [17]	✓	None	N/A	N/A	$O(D)$
	PDScanning [19]	✓	Monotonic Distance	N/A	N/A	$O(\min(D, D_{dis}^{\tau^2}))$
	PDScanning+ [4]	✓	Monotonic Distance	$O(ND^2)$	$O(D^2)$	$O(d)$
Hypothesis Testing	ADSampling [1]	×	Euclidean	$O(ND^2)$	$O(D^2)$	$O(\min(D, \frac{\tau^2}{(dis-\tau)^2} \log \frac{D}{\delta}))$
	DADE [2]	×	Euclidean	$O(ND^2)$	$O(D^2)$	$O(d)$
	DDCres [3]	×	Euclidean, Data, Query	$O(ND^2)$	$O(D^2)$	$O(d)$
Classification	DDCpca [3]	×	Euclidean, Index, Query	$O(ND^2)$	$O(D^2)$	$O(\sum_{j=1}^D \prod_{i=1}^{j-1} (1-p_i))$
	DDCpq [3]	×	Euclidean, Index, Query	$O(ND(D+c+2^b))$	$O(D^2 + D \cdot 2^b)$	$O(c + (1-p)D)$

¹ “Euclidean”: only applicable to Euclidean distance or distances that can be transformed into it.; “Data”: data vectors follow a known distribution (e.g., Gaussian); “Query”: query vectors follow the data distribution; “Index”: requires building a vector index for training.

² δ : failure probability for $(1+\epsilon)$ -approximation; b, c : product quantization parameters for subvector encoding length and codebook count.

E. Summary

Table III compares the existing DCO methods based on their exactness, assumption, and time complexity:

(1) **Exactness**: Simple scanning based algorithms are exact solutions, while others are approximate algorithms that trade minimal accuracy for efficiency enhancement. Moreover, only DADE provides an unbiased estimator for vector distances.

(2) **Assumptions**: DCO methods vary in their underlying assumptions. While most methods are restricted to Euclidean distance (or transformable metrics like inner product), PDScanning and PDScanning+ extend support to monotonic distances. DDCres, DDCpca, and DDCpq assume queries and data follow the same distribution. DDCres further assumes this distribution is Gaussian. Classification based methods require prior knowledge of the query parameter k and are tightly coupled to a specific index.

(3) **Time Complexity Breakdown**: Among existing methods, only FDScanning, PDScanning, and ADSampling provide an explicit analysis of their time complexity. However, ADSampling underestimates the $O(D^2)$ time cost to project the query vector into a new coordinate system. We classify this per-query operation as *online pre-processing*, a cost that also applies to PDScanning+ and all hypothesis testing based and classification based methods. Our work provides a comprehensive analysis on the time complexity by introducing d to denote the number of dimensions scanned and p_i as the probability of a negative prediction. We further account for the time cost of *offline pre-processing*, which includes computing PCA on the vector dataset and training the required models. Our analysis identifies **online pre-processing can become the dominant bottleneck at sufficiently high dimensions**, a finding that will be validated empirically later.

IV. BENCHMARK SETUP

This section introduces the detailed benchmark setup.

A. Overview

Our benchmark covers the following critical scenarios:

- **Query Processing**: Evaluating how DCOs impact vector similarity search performance across diverse datasets.
- **Index Construction**: The efficacy of DCOs in building mainstream vector indexes: HNSW [20] and IVF [21].
- **Diversified Hardware Condition**: The performance of DCOs under varied hardware configurations.
- **Common Parameters**: The impact of DCOs’ parameters.
- **Dynamic Updates**: The robustness and effectiveness of DCOs when handling dynamic data.
- **Out-of-Distribution (OOD) Queries**: Generalization when query vector distribution is different from the vector dataset (e.g., a multi-modal dataset).

TABLE IV: Dataset statistics

Dataset	Dim.	Category	Raw Data	Cardinality
Deep	96	Low-D	Image	100,000,000
GloVe	100	Low-D	Text	400,000
SIFT	128	High-D	Image	1,000,000
Text2Image	200	High-D	Multimodal	10,000,000
Laion	512	High-D	Multimodal	1,000,448
Wikipedia	768	High-D	Text	1,000,000
GIST	960	High-D	Image	1,000,000
OpenAI	1536	Ultra-High-D	Text	2,321,096
Trevi	4096	Ultra-High-D	Image	99,900
XUltr	12288	Ultra-High-D	Text	100,000

B. Dataset and Query Workload

Our benchmark employs 10 public datasets [22]–[24] with varying dimensions, sizes, and raw data types (see Table IV). They are all commonly used for evaluating vector similarity search. Most datasets contain in-distribution queries, which either share a distribution with the dataset or are sampled from it. In contrast, the multimodal datasets (Laion and Text2Image) exhibit an inherent distribution shift, as their data and query vectors are embedded from images and text, respectively, making their queries Out-of-Distribution (OOD).

To simulate the extremely ultra-high dimensionality of embeddings in modern LLMs (e.g., OpenAI’s davinci-001 model with 12,288 dimensions [25]), we also construct a synthetic dataset named XUltr. It is generated by concatenating token-level embeddings from the real-world dataset

MSMARCO [26] until the target dimensionality of 12,288 is reached. In this construction, each resulting vector represents the aggregation of tokens in a complete text phrase.

C. Compared DCO Algorithms and Their Implementations

We implement 8 DCO methods: FDScanning, PDScanning, PDScanning+, ADSampling, DADE, DDCres, DD-Cpca, and DDCopq [1]–[4], [17], [19]. The first three, categorized as simple scanning based methods, serve as *baselines*, since they are commonly used in industrial vector databases and vector search engines. The other five are the *state-of-the-art* algorithms (SOTA as short). To ensure a fair comparison, all algorithms are implemented within a unified framework with the following considerations:

- **PCA-based Pre-processing:** Methods using PCA for pre-processing are implemented with the NumPy library [27].
- **Index Integration:** Each method is integrated into two index-based vector retrieval solutions: HNSW (via hnswlib [20]) and IVF (via Faiss [17]). HNSW typically runs on CPUs, while IVF is often run on GPUs.
- **Target Hardware:** We target both CPU and GPU environments. On the CPU, we evaluate HNSW-based implementations with and without Intel SSE SIMD optimizations. On the GPU, we leverage massive parallelism in the IVF index by assigning one DCO operation per thread.
- **Parameter Settings:** We adopt the recommended parameter settings from the original implementations.
- **Distance Function:** All algorithms have been extended to support both inner product and Euclidean distance.

All algorithms are implemented in Python and C++, with the C++ components compiled with g++ 13.3.0 and -O3 optimization. SIMD instructions are enabled and multi-threading is disabled for CPU execution by default. We make our benchmark suite publicly available on GitHub [6].

D. Parameter Settings

The benchmark setup involves three types of parameters:

Index Parameters. We configure the graph-based index HNSW [20] with parameters $M = 16$ and $efConstruction = 500$, and the partition-based index IVF [21] with 4096 partitions, following the recommendations in Faiss [17].

Search Parameters. For HNSW, we vary the search depth $efSearch$ from 100 to 1500 in steps of 100. For IVF, we vary the number of probes $nprobe$ from 20 to 400 in steps of 20. The default settings are $efSearch = 200$ and $nprobe = 80$.

Query Parameter. We set the query parameter k to two common values, 20 and 100, as used in prior work [28]–[30].

E. Evaluation Metrics

We assess DCOs from the following three perspectives, where higher values indicate better performance:

- **Effectiveness:** Evaluated via **recall** defined in Def. 1.
- **Efficiency:** Measured by queries per second (QPS). As each query typically invokes the DCO multiple times, QPS represents the overall efficiency of DCOs.

- **Pruning Capability:** Quantified by the **dimension pruning ratio**, *i.e.*, the fraction of dimensions saved.

Following prior work [1]–[3], we plot QPS–recall curves by varying the index search parameters $efSearch$ and $nprobe$ for HNSW and IVF, respectively. Increasing these parameters raises recall while concurrently decreasing QPS.

F. Experimental Environment

We performed all experiments on a server with an AMD Ryzen 9 5950X CPU, 128GB RAM, and an NVIDIA GeForce RTX 3090 GPU (24GB VRAM). The server ran the Ubuntu 24.04.2 LTS with CUDA 12.4.

V. EXPERIMENTAL EVALUATION

This section presents our experimental results analysis. Due to page limits, we show only selected results here. Please refer to the full version [31] for the complete results.

A. Query Processing

This experiment evaluates the performance improvement from applying DCOs to vector similarity search for $k = 20$ and $k = 100$. We use several datasets with varying dimensions (96 to 12,288) and cardinalities (99,900 to 100 million) to ensure a comprehensive assessment. Fig. 4 presents time–recall curves to demonstrate the effectiveness of DCO methods.

Overall Query Performance. The results show that in only 4 out of the 12 subfigures, the hypothesis testing based and classification based methods consistently outperform the naive baseline FDScanning. Specifically, compared to FDScanning, the hypothesis testing based methods improve QPS by up to $1.4\text{--}1.9\times$, while the classification based methods (DDCpca and DDCopq) achieve $1.6\text{--}2.1\times$ improvements. These results demonstrate the superior enhancement in time efficiency when applying DCOs to vector similarity search, while their recall remains largely stable with at most 2% change.

We also observe cases where the baselines, specifically FDScanning, perform better than the others. For instance, Fig. 4a and 4c show that FDScanning always achieves the highest QPS on the Deep dataset. Similarly, Fig. 4j and 4l show that PDScanning obtains the optimal efficiency for recall lower than 94% on the Trevi dataset.

The SOTA methods become ineffective on the Deep and Trevi datasets for different reasons. On the Deep dataset, which has the lowest dimensionality, the time saved by skipping 6%–36% of dimensions is marginal (except for DDCopq). On the Trevi dataset, which has very high dimensionality, the $O(D^2)$ online pre-processing time per query vector becomes the efficiency bottleneck. This cost is not amortized effectively when the recall is below 94%, because DCOs are only invoked for $O(\log N)$ candidate vectors when $efSearch$ is small on HNSW [20] for vector similarity search.

This phenomenon becomes increasingly pronounced at higher dimensionalities. Fig. 5 shows the result on the XULtra dataset which has the highest dimensionality. Here, FDScanning and PDScanning achieve higher QPS than all SOTA

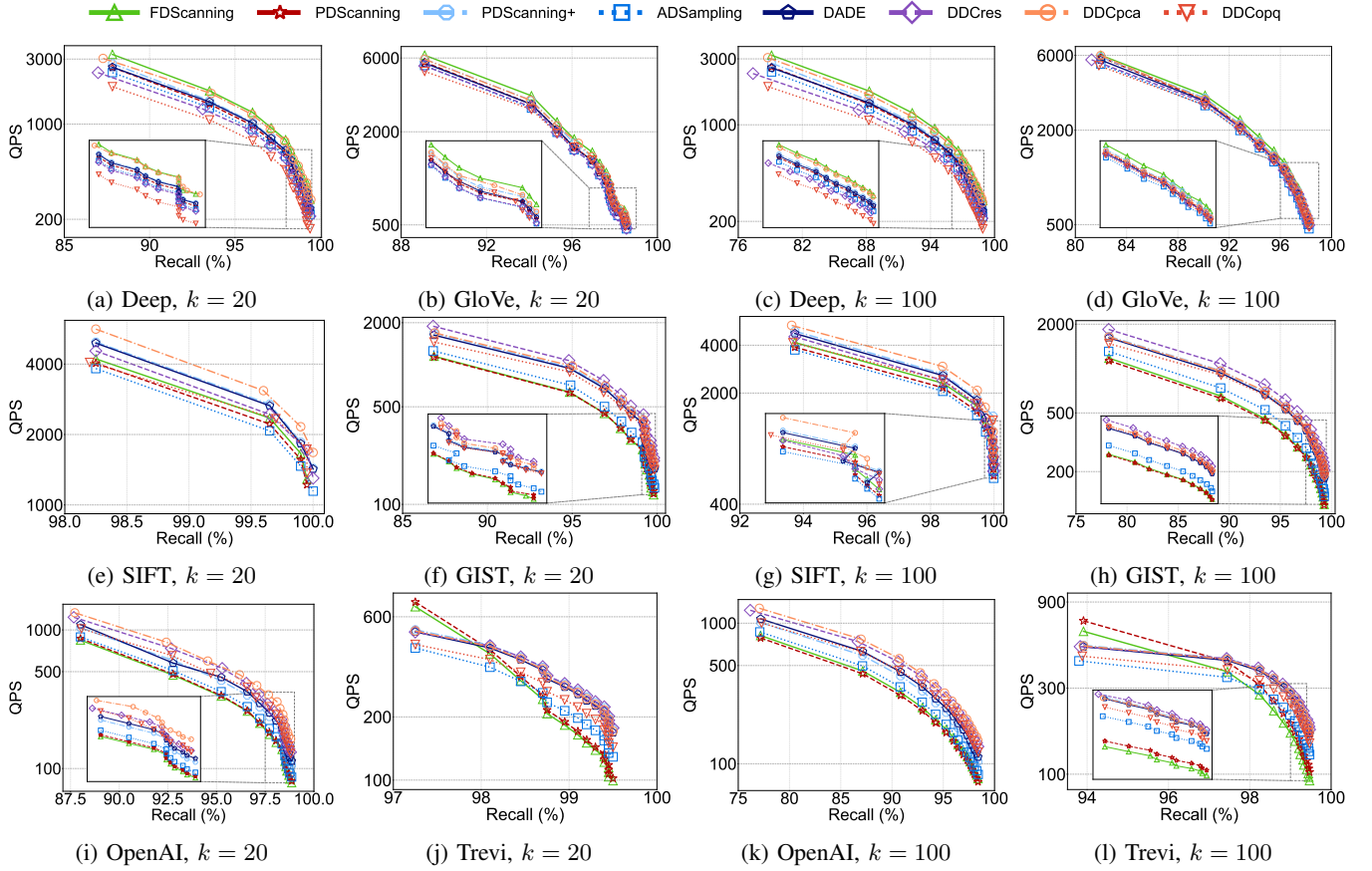


Fig. 4: Query performance when DCOs are applied to vector similarity search

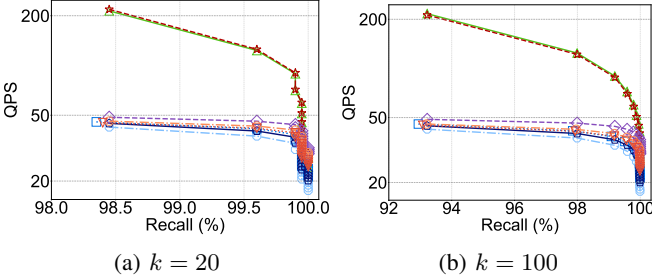


Fig. 5: Performance on ultra-high-dimensional dataset XULtra

methods for both k values. The QPS of these SOTA methods decreases by up to $0.8\text{--}0.8\times$ compared to FDScanning.

Pruning Capability of DCO. Fig. 6 illustrates the dimension pruning ratios of DCOs and their recall across three datasets of varying dimensionality. The results demonstrate that the pruning capabilities of DCOs are dimension-dependent: aside from DDCopq, DDCpca achieves the highest pruning ratio on the Deep dataset, while DDCres performs best on the other two datasets with higher dimensions. On Deep, the pruning capability of ADSampling is nearly equivalent to that of PDScanning, which is consistent with its suboptimal performance in Fig. 4. Moreover, all methods maintain a

comparable level of recall. This consistent result indicates that the minor approximation errors of the DCO solutions do not compromise the query accuracy.

Takeaway #1: DCO is no silver bullet. While adept at skipping unnecessary dimensions with minimal recall loss, DCOs do not universally boost efficiency. As our results on the Deep and XULtra datasets show, they can even be slower than the brute-force baseline FDScanning, especially on low- and ultra-high-dimensional datasets.

B. Out-of-Distribution Queries

Out-of-distribution (OOD) queries are commonly seen in real-world applications like multimodal retrieval. To evaluate the robustness of DCOs under this setting, we conduct experiments on two text-to-image datasets: Laion and Text2Image. For comparison, we also include in-distribution queries by sampling query vectors directly from the image embeddings.

In Fig. 7, simple scanning based baselines, FDScanning and PDScanning, consistently achieve higher QPS than most SOTA methods on OOD queries. While DDCpca achieves up to $1.5\times$ higher QPS than FDScanning on in-distribution queries, its performance degrades significantly on OOD queries, becoming up to $1.6\times$ slower than it. The only

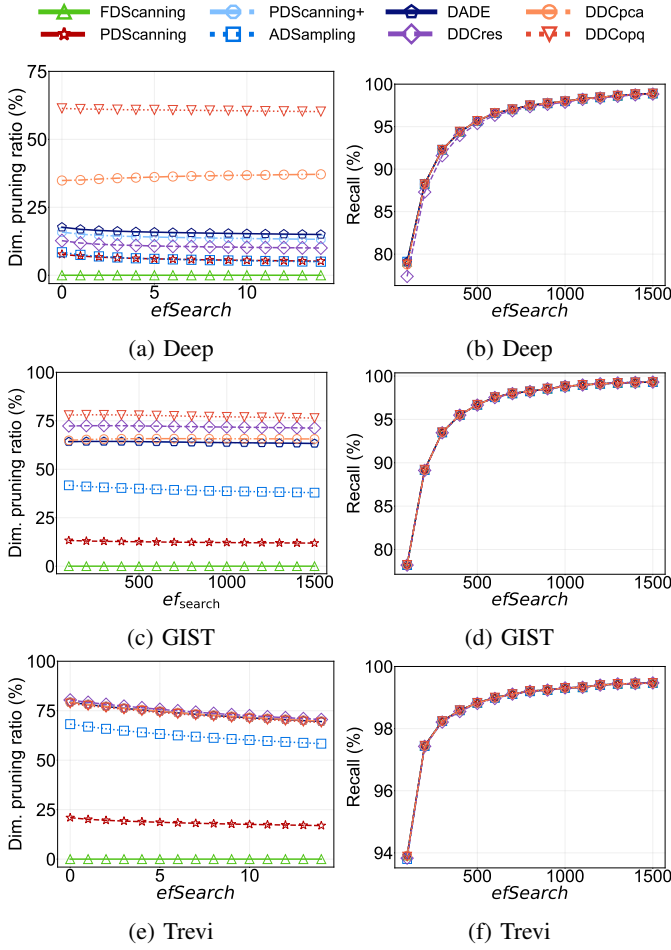


Fig. 6: Dimension pruning via DCO and its impact on recall

exception is DDCopq, which achieves the highest QPS on the Laion dataset. The superior performance of DDCopq here is primarily driven by its high prediction accuracy (over 95% on Laion).

Furthermore, even for in-distribution queries, FDS Scanning and PD Scanning maintain higher QPS than PD Scanning+, ADSampling, DADE, DDCres and DDCopq in Fig. 7c. This result further confirms the sensitivity of existing DCO methods to the data dimensionality.

Takeaway #2: SOTA methods often fail to maintain efficiency gains on OOD queries. In particular, none of these SOTA algorithms achieves higher QPS than FDS Scanning across both datasets.

C. Beyond Euclidean Distance

This experiment evaluates the extension of DCOs to non-Euclidean distance, specifically the inner product (IP), as all DCOs support it. We use the Wikipedia and XULtra datasets in Table IV, which natively employ IP, alongside two other datasets whose vectors were pre-normalized for IP-based vector search.

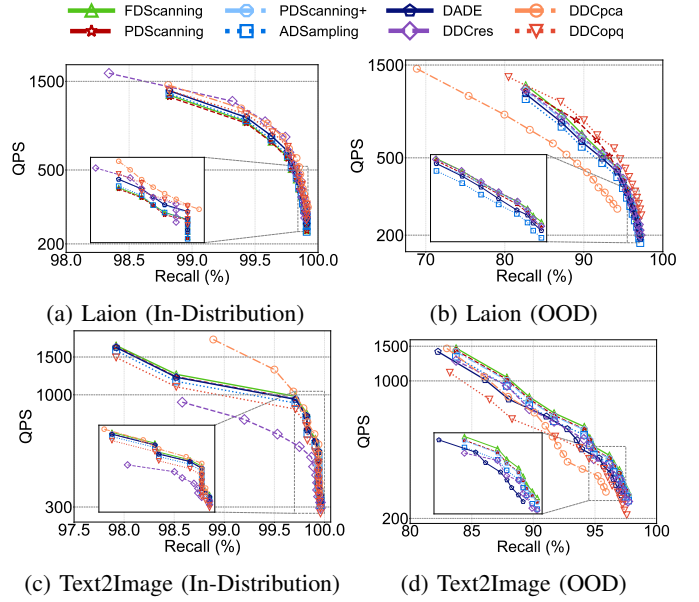


Fig. 7: Results on in-distribution and OOD queries

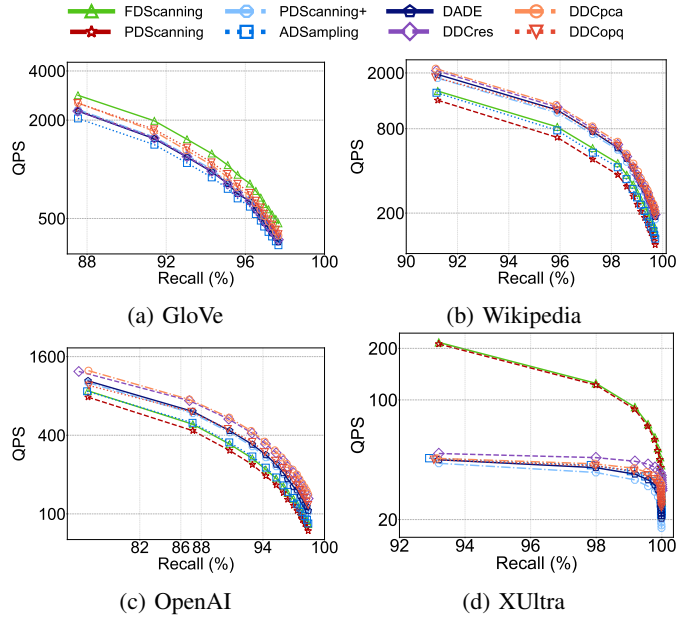


Fig. 8: Performance on different metrics

The similar trends observed in Fig. 8 and Fig. 4 are expected, as most DCO methods convert IP into a Euclidean distance computation. On the OpenAI dataset, DADE, DDCres, DDCpca, and DDCopq improve QPS by up to 1.3–1.7 $\times$ compared to FDS Scanning, which demonstrates their ability to handle IP. However, there are also cases on GloVe and XULtra datasets, where FDS Scanning outperforms all the other methods. For instance, it achieves the highest QPS on GloVe, outperforming the SOTA methods by up to 1.2–1.4 $\times$. Meanwhile, the bottleneck caused by online preprocessing time persists: the QPS of these SOTA methods drops by up to

0.8 $\times$ compared to FDScanning.

These phenomena indicate that DCOs have similar performance in both Euclidean distance and IP. This can be attributed to the fact that most DCO algorithms transform IP into Euclidean distance to generalize across distance metrics.

Takeaway #3: The similar performance of DCOs under IP and Euclidean metrics stems from their reliance on converting the former to the latter, but this comes at the cost of generality.

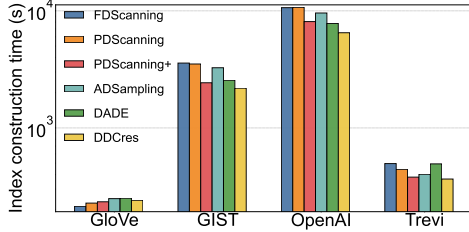


Fig. 9: Index construction time using DCO

D. Index Construction

While most DCOs have the potential to accelerate index construction, this capability has not been systematically evaluated in prior work. We fill this gap by evaluating their performance, excluding the classification based methods (DDCpca and DDCopq) that themselves require an index for training.

Index Construction Time. Fig. 9 shows the index construction time with DCOs, which includes the offline pre-processing time of DCOs. On the low-dimensional dataset GloVe, construction time even increases with the SOTA methods. By contrast, on high-dimensional datasets (GIST and OpenAI), most DCO methods achieve acceleration, including PDScanning+, ADSampling, DADE, and DDCres. For example, DDCres reduces index construction time by up to 0.4 $\times$ across these datasets. This further confirms that most DCO methods do not perform well in datasets with comparably lower dimensions. On the ultra-high-dimensional dataset Trevi, ADSampling and DADE can also reduce the construction time, but are still longer than PDScanning+. The pre-processing time for PCA is no longer the bottleneck, because the HNSW construction dominates the total time cost.

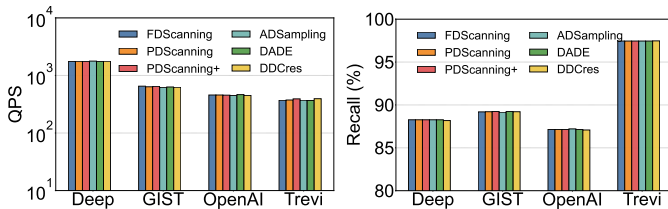


Fig. 10: Performance evaluation of different indices

Vector Similarity Search Performance. As most DCOs are approximate, they may result in different indexes. We

evaluate the constructed indexes on vector similarity search using a fixed query processing method (FDScanning) for a fair comparison. Fig. 10 shows that all indexes achieve nearly identical average recall and QPS. This demonstrates that indexes built with DCOs effectively maintain end-to-end vector search performance.

Takeaway #4: DCOs can accelerate vector index construction without compromising search performance, but their efficacy is dependent on data dimensionality.

E. Robustness on Dynamic Data

While DCO methods have demonstrated strong performance on certain static datasets, their behavior in dynamic data remains unknown due to limited evaluations on this scenario. We fill this gap by conducting experiments on incrementally inserting data. We ignore the experiment of deletions, since deletions are often handled by a flag in conventional vector indexes, which has limited impact on DCO's performance.

Stability Under Sequential Data Insertions. We first examine how DCO methods behave when new vectors are inserted into the GIST dataset. Starting from an HNSW index built on 60% of the dataset, we perform four rounds of insertions, each adding 10% more data. Fig. 11 reports the QPS and recall after each insertion round with the corresponding total update time.

During insertion, most methods achieve acceleration compared to FDScanning. For instance, DDCres reduce the update time by up to 0.4 $\times$. Meanwhile, as the dataset grows, the recall of all methods declines due to the increased search space, while DCO methods exhibit similar recall. After updating, the QPS of some methods decreases; however, the SOTA methods still outperform FDScanning. This suggests that DCOs can speed up vector insertion without degrading search performance.

Performance with Limited Initial Dataset. Classification based methods, DADE and PDScanning+ rely heavily on the initial data for training prediction models or computing PCA matrices. To evaluate their sensitivity, we test their performance across a wide range of initial dataset sizes (*i.e.*, from 0.01% to 100% of the full dataset) and report the resulting recall and dimension pruning ratio in Fig. 12.

Specifically, with only 1% of the full data as the training set, PDScanning+, DADE, and DDCres achieve effective pruning performance. They demonstrate strong adaptability to real-world scenarios, where only a small fraction of data is available at the start and more is appended over time. In contrast, DDCopq and DDCpca require at least 5% of the full dataset to reach a relatively stable recall and pruning ratio. Moreover, both methods require a minimum of 10,000 vectors for training, which explains why their results are absent when the initial dataset is below 1%. It is worth noting that the pruning ratio of DDCpca keeps increasing as the initial dataset size grows.

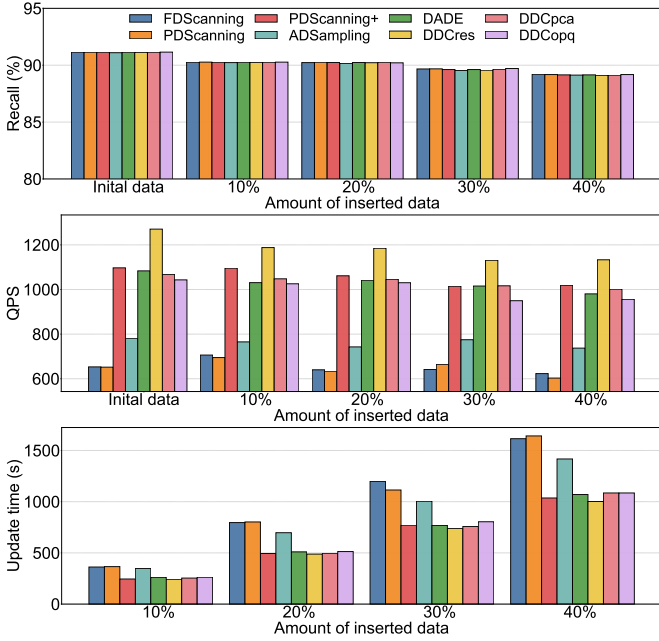


Fig. 11: QPS, recall and update time of DCO methods as new vectors are incrementally inserted into the GIST dataset

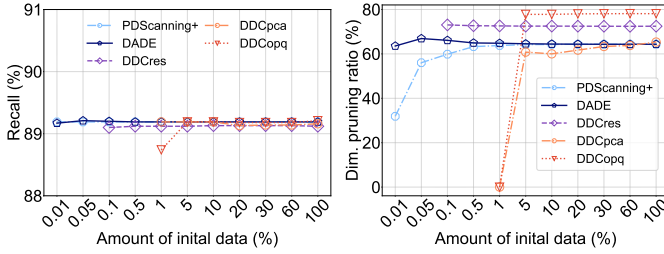


Fig. 12: Performance with limited initial data

Takeaway #5: DCOs perform well on dynamic data and can accelerate vector insertion, whereas classification based methods require sufficient initial data for training.

F. Diversified Hardware Condition

We evaluate DCO methods across different hardware platforms: GPU and CPU (with/without SIMD support). Consequently, we use the IVF index for this experiment, as HNSW often lacks native GPU support.

Performance on CPU with and without SIMD Support. As illustrated in the first two rows of Fig. 13, the usage of SIMD instructions significantly alters the performance ranking of DCO methods in terms of QPS. For example, when achieving over 92% recall on the SIFT dataset, DDCopq is the least efficient method with SIMD disabled, but becomes the most efficient when SIMD is enabled. This is because SIMD enables the parallel computation of multiple quantization distances by packing them into a single instruction. The simple scanning based method PDScanning+ consistently maintains a high

QPS on the Trevi dataset, where the QPS gap to the optimal one is usually within $0.01\times$ for recall above 89%.

Another finding is that SIMD instructions provide a substantial QPS boost for all methods. For example, FDScanning exhibits a QPS increase of up to $3.3\times$ on the GIST dataset when using SIMD. Given that modern AMD and Intel processors usually support SIMD instruction sets like SSE and AVX [32], this improvement is almost a “free lunch”. This also justifies our benchmark to enable SIMD as the default setting. However, enabling SIMD instructions narrows the efficiency advantage of DCO methods over FDScanning, reducing the speedup from $2.0\text{--}4.4\times$ to $1.2\text{--}3.0\times$ on the IVF index¹.

Performance on GPU. The third row of Fig. 13 plots the QPS-recall curves of DCO methods running on GPU. Most DCO methods exhibit similar performance on the GloVe dataset, while DDCres and DDCpca underperform the others. For other datasets at recall above 73%, PDScanning+, DADE, DDCres, and DDCpca achieve similar QPS across datasets, often outperforming the others. PDScanning performs comparably better than ADSampling. The results of PDScanning and PDScanning+ confirm that simple scanning based methods remain competitive with GPU acceleration. Overall, ADSampling, DADE, DDCres, DDCpca, and DDCopq improve QPS with the same recall over FDScanning by up to $4.9, 7.0, 7.6, 6.9,$ and $4.7\times$ respectively, across these datasets.

On the ultra-high dimensional dataset Trevi, most methods have higher QPS than FDScanning. This result is inconsistent with earlier findings in Fig. 4. The reason lies in the IVF index processing many more candidates than HNSW, with each candidate requiring a DCO. As a result, the time spent on online pre-processing query vectors is dwarfed by the total time of DCOs and thus does not create a bottleneck.

Performance Comparisons: CPU vs GPU. Compared to CPU without SIMD, DCO methods can improve their QPS by up to $3.0\text{--}5.0\times$ on GPU. This improvement benefits from vector-level parallelization, which allows concurrent DCOs to be performed for a batch of query vectors. However, this advantage drops to roughly $1.1\text{--}2.1\times$, when compared to CPU with SIMD enabled. Based on these comparison results, we have two major observations: (1) GPU is preferable for evaluating the performance of DCOs on IVF, as it delivers the best performance, and (2) HNSW is better suited to CPU with SIMD, since it does not natively support GPU execution.

Takeaway #6: Hardware configuration is a critical yet overlooked factor in DCO evaluation. Our experiment shows that hardware selection can drastically alter performance rankings. For instance, enabling SIMD on CPU can elevate DDCopq from the least to the most efficient method on the SIFT dataset.

G. Summary: Recommendations, Merits, and Limitations

¹The speedup reduces from $1.9\text{--}3.1\times$ to $1.1\text{--}2.1\times$ on HNSW (see [31]).

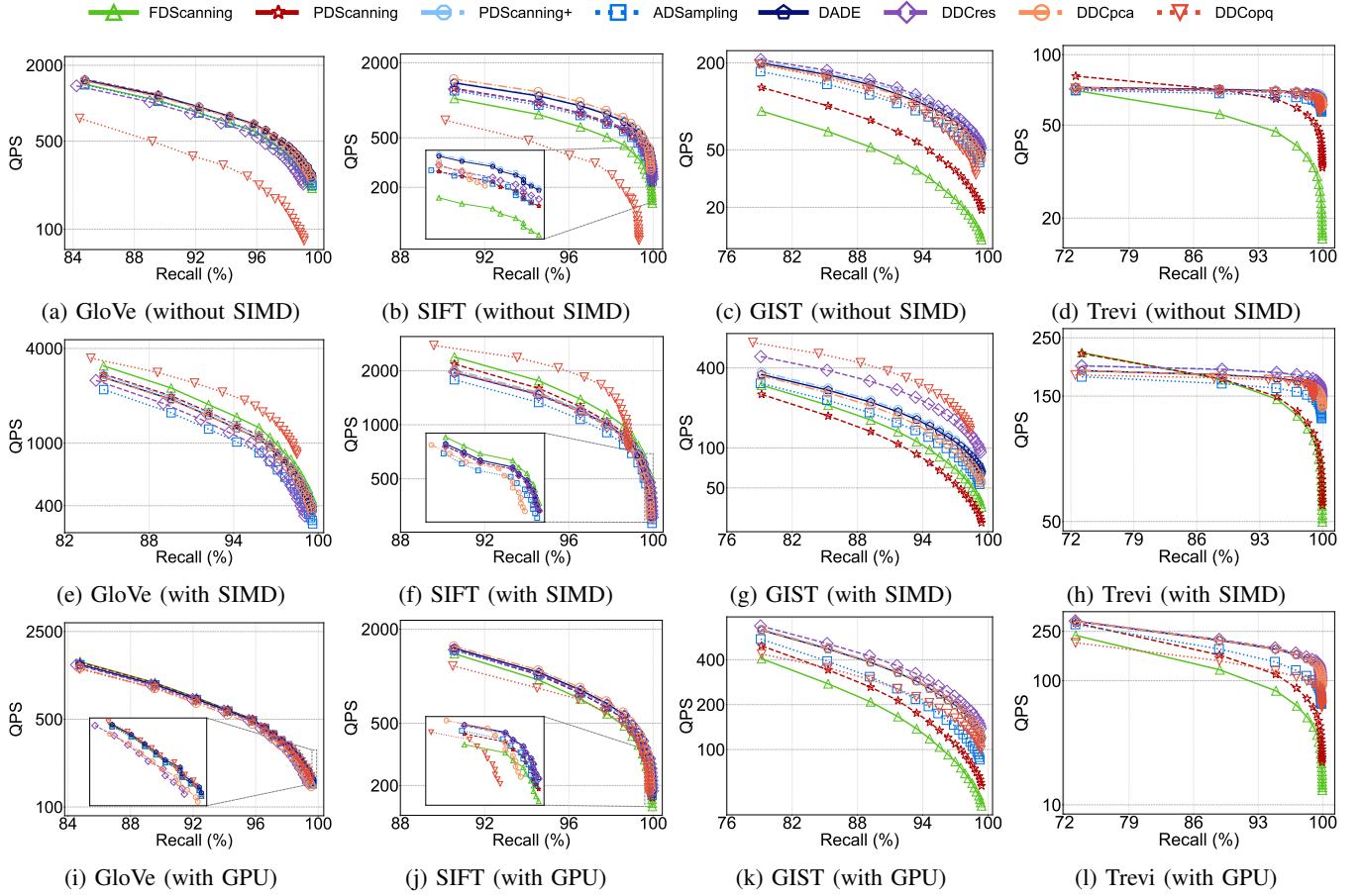


Fig. 13: Comparison of DCO methods on the IVF index across different hardware environments

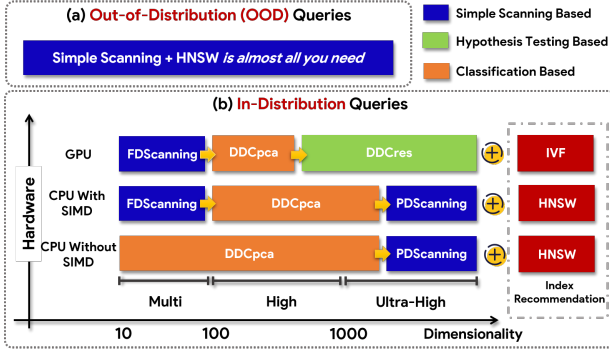


Fig. 14: Recommendations of DCO methods

Overall Recommendation. Our experimental results lead to a clear set of recommendations for selecting the optimal DCO method in different scenarios, as shown in Fig. 14. These recommendations are determined by each method’s average QPS ranking for achieving a recall rate over 95%.

The optimal DCO algorithm selection depends on three primary factors: *data dimensionality*, *hardware configurations*, and *query distribution*. For example, on CPUs with SIMD support, the best choice for in-distribution queries shifts from

FDScanning to DDCpca, and then to PDScanning, as data dimensionality increases. Overall, simple scanning based methods (FDScanning and PDScanning) outperform the others for OOD queries, and excel on low- and ultra-high-dimensional data. For in-distribution queries, the classification based method DDCpca is often optimal on high-dimensional data, whereas the hypothesis testing based method DDCres performs particularly well with the IVF index on GPUs.

Based on the above recommendations, we next analyze the main merits and limitations of existing DCO methods to guide their practical usage and future research.

Merits. Existing DCO methods offer several key advantages:

- **Enhanced Query Throughput.** These methods can substantially improve the query throughput of vector similarity search. For instance, on a CPU with SIMD enabled, PDScanning+, ADSampling, DADE, DDCres, DDCpca, and DDCopq improved QPS by up to 1.9, 1.4, 1.8, 1.9, 2.1, and 1.6× on HNSW, respectively.
- **Native Recall Preservation.** Most DCO methods preserve the native recall of underlying vector indexes across various scenarios, despite their approximate nature. For instance, hypothesis testing based methods exhibit a recall variation consistently below 2% in our experiments.

- **Accelerated Index Construction.** Several DCO methods can accelerate vector index building, but primarily on high- and ultra-high-dimensional datasets.
- **Extensibility to Other Distance Metrics.** Although primarily designed for Euclidean distance, these algorithms can be adapted to other prevalent metrics while maintaining recall, such as inner product and cosine similarity.

Limitations. We also identify the following key limitations:

- **No Silver Bullet for Universal Efficiency.** The efficiency advantages of these methods are not universal but highly dependent on data dimensionality. On low- and ultra-high-dimensional datasets, most SOTA methods are slower than simple scanning based baselines, with a QPS reduction of up to $0.1\text{--}0.4\times$ and $0.8\text{--}0.8\times$ respectively.
- **Vulnerability to Out-of-Distribution (OOD) Queries.** Their performance is not robust to distribution shifts of query vectors. Experiments on multi-modal datasets (Laion and Text2Image) show that simple scanning based methods (e.g., FDScanning) can be more efficient than several SOTA methods, including ADSampling, DADE, DDCres, and DDCpca.
- **Dependence on Hardware Environment.** The efficiency gains of DCO methods are contingent on hardware. When SIMD enabled, the QPS of all methods improves, but their advantages over FDScanning narrow significantly, dropping from $2.0\text{--}4.4\times$ to $1.2\text{--}3.0\times$ on IVF, and from $1.9\text{--}3.1\times$ to $1.1\text{--}2.1\times$ on HNSW (see our full paper [31]).
- **Restrictive Assumptions Limit General Applicability.** Most DCO methods rely on specific assumptions about data distributions, query workloads, or distance metrics. For example, despite being a top performer in several scenarios, DDCpca requires prior knowledge of the query workload and parameter k , sufficient training data, and is primarily designed for Euclidean and IP distances.

VI. RELATED WORK

We review the related work from the following two aspects: *vector similarity search* and *vector distance operation*.

Vector Similarity Search. Vector similarity search is broadly categorized into *exact* and *approximate* solutions. While exact solutions guarantee correctness, they often suffer from high query latency on large-scale datasets due to the curse of dimension [15]. Consequently, recent work has focused primarily on approximate algorithms [33]–[35]. A common optimization technique in these algorithms is the usage of specialized indexes, which generally fall into three categories:

- **Graph-based indexes** (e.g., HNSW [20], HVS [36], and SHG [37]) organize vectors into a graph (or a hierarchy of graphs), where each node represents a vector and is connected by edges to sufficiently nearby vectors.
- **Partition-based indexes** (e.g., IVF [21] and Ada-IVF [38]) divide the vector space into multiple shards, enabling many shards to be pruned during query processing. One of the most popular partitioning strategy is high-dimensional vector clustering [39].

- **Hash-based indexes**, such as QALSH [40] and PMLSH [41], employ locality-sensitive hashing (LSH) [42] to map similar vectors into the same hash bucket, thereby narrowing the search space.

In practice, graph-based and partition-based indexes are the most widely adopted in production vector databases [43], [44] and search engines [17], [45]. Typical examples include HNSW [20] and IVF [21]. HNSW is predominantly CPU-oriented, while IVF is more commonly accelerated on GPUs [16], [46]. Recent studies also explore hybrid models, such as APG-LSH [47], which integrates both graph and LSH to leverage their complementary strengths.

Vector similarity search in indexing-based solutions typically adopts a filter-and-refine framework. During query processing, a large number of *vector distance operations* are performed. Besides the standard distance computation, several other operations are commonly used:

Vector Distance Operation. Query processing in these indexing-based solutions involves numerous *vector distance operations*. Beyond the standard *distance computation operation*, several other operations are commonly used:

- **Comparison.** Distance Comparison Operation (DCO) compares the distance between a data vector and a query vector against a given threshold.
- **Quantization.** Quantization reduces the dimensionality of a high-dimensional vector and enables faster (approximate) distance computation.
- **Fusion.** This operation integrates vector embeddings and their associated attributes into a single distance function [48], [49]. It is often applied within a hybrid data model that combines vector and relational data.

Among these operations, DCO is our primary concern. We also evaluate a DCO method, DDCopq, which utilizes Product Quantization (PQ) for acceleration. Although straightforward, the DCO methods, FDScanning and PDScanning, are commonly employed in vector databases and search engines [17], [19], [20], [43], [50], [51]. PDScanning has also been applied in approximate nearest neighbor search in near-data processing architecture [5]. PDScanning+ [4] applies PCA in PDScanning to prioritize dimensions with greater impact on distances. While Gao *et al.* [1] first introduced hypothesis testing to support DCOs, Deng *et al.* [2] and Yang *et al.* [3] later built upon this technique to propose more effective solutions. Furthermore, Yang *et al.* [3] formulated DCO as a classification problem, and developed two classification based methods: DDCpca and DDCopq.

VII. CONCLUSION AND FUTURE DIRECTION

Distance comparison operation (DCO) has emerged as a promising method for accelerating vector similarity search. While prior work has introduced several effective DCO algorithms, their evaluations have often been conducted in restricted settings, failing to expose their limitations under broader yet realistic conditions. To bridge this gap, we have conducted a comprehensive benchmark on eight state-of-the-art (SOTA) methods across ten diverse datasets. Our evaluation

demonstrates that *no current method represents a silver bullet*: for out-of-distribution (OOD) queries or data with dimensionality outside the 100-1000 range, SOTA methods can even increase query latency compared to naive baselines.

Motivated by these observations, we identify the following research directions for future work:

- (1) Developing specialized optimization methods tailored to relatively low- or ultra-high-dimensional datasets.
- (2) Improving robustness against OOD queries, which are prevalent in real-world applications like multi-modal retrieval.
- (3) Incorporating hardware configuration (*e.g.*, GPU and SIMD) as a primary design consideration.
- (4) Extending the scope of DCOs beyond Euclidean metrics and other limiting assumptions to enhance generality.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

During the preparation of this work, we used DeepSeek-R1 in order to assist with proofreading and correction of grammatical errors and typos throughout the entire manuscript. This AI tool was applied to main sections of the article at a superficial, non-substantive level to improve readability. The authors reviewed and edited all AI-suggested changes and take full responsibility for the content of the manuscript.

REFERENCES

- [1] J. Gao and C. Long, “High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations,” *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 137:1–137:27, 2023.
- [2] L. Deng, P. Chen, X. Zeng, T. Wang, Y. Zhao, and K. Zheng, “Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 18, no. 3, pp. 812–821, 2024.
- [3] M. Yang, W. Li, J. Jin, X. Zhong, X. Wang, Z. Shen, W. Jia, and W. Wang, “Effective and general distance computation for approximate nearest neighbor search,” in *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*, 2025, pp. 1098–1110.
- [4] Z. Wang, H. Xiong, Q. Wang, Z. He, P. Wang, T. Palpanas, and W. Wang, “Dimensionality-reduction techniques for approximate nearest neighbor search: A survey and evaluation,” *IEEE Data Eng. Bull.*, vol. 48, no. 3, pp. 63–80, 2024.
- [5] Y. Li, Y. Jin, B. Tian, H. Zhang, and M. Gao, “ANSMET: approximate nearest neighbor search with near-memory processing and hybrid early termination,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture, ISCA 2025, Tokyo, Japan, June 21-25, 2025*. ACM, 2025, pp. 1093–1107.
- [6] “Source code and dataset in the benchmark.” [Online]. Available: https://github.com/zzlin237/DCO_COMPARE.git
- [7] G. Li, J. Sun, J. Pan, J. Wang, Y. Xie, R. Liu, and W. Nie, “Gaussdb-vector: A large-scale persistent real-time vector database for LLM applications,” *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 4951–4963, 2025.
- [8] Y. Han, C. Liu, and P. Wang, “A comprehensive survey on vector database: Storage and retrieval technique, challenge,” *CoRR*, vol. abs/2310.11703, 2023.
- [9] G. Casella and R. Berger, *Statistical inference*. Chapman and Hall/CRC, 2024.
- [10] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An introduction to statistical learning: with applications in R*. Springer, 2013, vol. 103.
- [11] R. Szeliski, *Computer vision: algorithms and applications*. Springer Nature, 2022.
- [12] C. Manning and H. Schütze, *Foundations of statistical natural language processing*. MIT press, 1999.
- [13] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, “Recent advances in natural language processing via large pre-trained language models: A survey,” *ACM Comput. Surv.*, vol. 56, no. 2, pp. 30:1–30:40, 2024.
- [14] B. Pecher, I. Srba, and M. Bieliková, “A survey on stability of learning with limited labelled data and its sensitivity to the effects of randomness,” *ACM Comput. Surv.*, vol. 57, no. 1, pp. 19:1–19:40, 2025.
- [15] C. D. Toth, J. O’Rourke, and J. E. Goodman, *Handbook of discrete and computational geometry*. CRC press, 2017.
- [16] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [17] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” *CoRR*, vol. abs/2401.08281, 2024.
- [18] P. H. Chen, W. Chang, J. Jiang, H. Yu, I. S. Dhillon, and C. Hsieh, “FINGER: fast inference for graph-based approximate nearest neighbor search,” in *WWW*, 2023, pp. 3225–3235.
- [19] M. Muja and D. G. Lowe, “Fast approximate nearest neighbors with automatic algorithm configuration,” in *VISAPP 2009 - Proceedings of the Fourth International Conference on Computer Vision Theory and Applications, Lisboa, Portugal, February 5-8, 2009 - Volume 1*, 2009, pp. 331–340.
- [20] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
- [21] A. Babenko and V. S. Lempitsky, “The inverted multi-index,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 6, pp. 1247–1260, 2015.
- [22] “Big ANN Benchmark,” 2024. [Online]. Available: <https://big-ann-benchmarks.com/>
- [23] “YouTube-8M: A Large and Diverse Labeled Video Dataset for Video Understanding Research.” [Online]. Available: <https://research.google.com/youtube8m/download.html>
- [24] “GloVe: Global Vectors for Word Representation.” [Online]. Available: <https://nlp.stanford.edu/projects/glove/>
- [25] “New and improved embedding model,” <https://openai.com/index/new-and-improved-embedding-model/>.
- [26] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, “MS MARCO: A human generated machine reading comprehension dataset,” in *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016*, ser. CEUR Workshop Proceedings, vol. 1773, 2016.
- [27] C. R. Harris, K. J. Millman, S. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with numpy,” *Nat.*, vol. 585, pp. 357–362, 2020.
- [28] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, “Diskann: Fast accurate billion-point nearest neighbor search on a single node,” *NeurIPS*, vol. 32, 2019.
- [29] M. Wang, X. Xu, Q. Yue, and Y. Wang, “A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 1964–1978, 2021.
- [30] H. Zhang, Y. Wang, Q. Chen, R. Chang, T. Zhang, Z. Miao, Y. Hou, Y. Ding, X. Miao, H. Wang, B. Pang, Y. Zhan, H. Sun, W. Deng, Q. Zhang, F. Yang, X. Xie, M. Yang, and B. Cui, “Model-enhanced vector index,” in *Neural*, 2023.
- [31] “Distance comparison operations are not silver bullets in vector similarity search: An experimental study on their merits and limits (full paper).” [Online]. Available: https://github.com/zzlin237/Metric_and_Limit_of_DCOs.git
- [32] P. Guide, “Intel® 64 and ia-32 architectures software developer’s manual,” *Volume 3B: system programming guide, Part*, vol. 2, no. 11, pp. 0–40, 2011.
- [33] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, “Approximate nearest neighbor search on high dimensional data - experiments, analyses, and improvement,” *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1475–1488, 2020.
- [34] J. Qin, W. Wang, C. Xiao, and Y. Zhang, “Similarity query processing for high-dimensional data,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3437–3440, 2020.
- [35] J. Qin, W. Wang, C. Xiao, Y. Zhang, and Y. Wang, “High-dimensional similarity query processing for data science,” in *KDD ’21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18, 2021*, F. Zhu, B. C. Ooi, and C. Miao, Eds., 2021, pp. 4062–4063.
- [36] K. Lu, M. Kudo, C. Xiao, and Y. Ishikawa, “HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 15, no. 2, pp. 246–258, 2021.
- [37] Z. Gong, Y. Zeng, and L. Chen, “Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts,” *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3518–3530, 2025.
- [38] J. Mohoney, A. Pacaci, S. R. Chowdhury, U. F. Minhas, J. Pound, C. Renggli, N. Reyhani, I. F. Ilyas, T. Rekatsinas, and S. Venkataraman, “Incremental ivf index maintenance for streaming vector search,” *arXiv preprint arXiv:2411.00970*, 2024.
- [39] Y. Fu, C. Chen, X. Chen, W. Wong, and B. He, “Optimizing the number of clusters for billion-scale quantization-based nearest neighbor search,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6786–6800, 2024.
- [40] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng, “Query-aware locality-sensitive hashing for approximate nearest neighbor search,” *Proc. VLDB Endow.*, vol. 9, no. 1, pp. 1–12, 2015.

- [41] B. Zheng, X. Zhao, L. Weng, Q. V. H. Nguyen, H. Liu, and C. S. Jensen, "PM-LSH: a fast and accurate in-memory framework for high-dimensional approximate NN and closest pair search," *VLDB J.*, vol. 31, no. 6, pp. 1339–1363, 2022.
- [42] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *STOC*, 1998, pp. 604–613.
- [43] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, "Milvus: A purpose-built vector data management system," in *SIGMOD*. ACM, 2021, pp. 2614–2627.
- [44] "Weaviate." [Online]. Available: <https://github.com/weaviate/weaviate>
- [45] "Qdrant." [Online]. Available: <https://github.com/qdrant/qdrant>
- [46] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, 2020, pp. 39–48.
- [47] X. Zhao, Y. Tian, K. Huang, B. Zheng, and X. Zhou, "Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces," *Proc. VLDB Endow.*, vol. 16, no. 8, pp. 1979–1991, 2023.
- [48] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An efficient and robust framework for approximate nearest neighbor search with attribute constraint," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- [49] A. A. Aomar, K. Echihabi, M. Arnaboldi, I. Alagiannis, D. Hilloulin, and M. Cherkaoui, "Rwalks: Random walks as attribute diffusers for filtered vector search," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 212:1–212:26, 2025.
- [50] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: postgresql ultra-high-dimensional approximate nearest neighbor search extension," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, 2020, pp. 2241–2253.
- [51] K. Figueroa, G. Navarro, and E. Chavez, "The metric spaces library maintained by the sisap initiative," 2017. [Online]. Available: <https://github.com/kaarinita/metricSpaces>

A. Query Processing without SIMD on HNSW

This experiment evaluates the performance improvement from applying DCOs to vector similarity search on HNSW with SIMD disable.

Overall Query Performance without SIMD. Compared to the result in Fig. 4, the rankings of different DCO methods have undergone dramatic changes when SIMD is disable. Specifically, FDScanning no longer achieves the best performance on the Deep dataset. Meanwhile, compared to FDScanning, hypothesis testing based methods improve QPS by up to $1.9\text{--}2.5\times$, while the classification based methods achieve $2.1\text{--}3.1\times$ improvements. These advantages over FDScanning are larger than those observed when SIMD is enabled. This phenomenon can be attributed to the fact that SIMD primarily accelerates the distance computation itself, while the overhead of the termination decision logic, *i.e.*, determining whether to terminate computation unchanged. Consequently, the relative benefit of DCOs is diluted when SIMD is active, as the non-computational components dominate a larger fraction of the total runtime.

We also observe cases where the baseline PDScanning outperforms other methods. As shown in Fig. 15j and Fig. 15k, PDScanning achieves the highest QPS when the recall is below 97.5%. This demonstrates that the online preprocessing time $O(N^2)$ is also a bottleneck for the SOTA methods for some ultra-high-dimensional datasets on HNSW when SIMD is disable.

B. Out-of-Distribution Queries without SIMD

To further evaluate the robustness of DCOs under out-of-distribution queries, we also conduct this experiment with SIMD disabled.

As shown in Fig. 16a and Fig. 16c, DDCpca achieves the highest QPS on both datasets for in-distribution queries. However, it becomes the least efficient method under OOD queries, performing up to $1.6\times$ and $1.8\times$ slower than FDScanning and PDScanning, respectively. In this setting, aside from DDCopq, which achieves the highest QPS on the Laion dataset, the loop-based baselines PDScanning and PDScanning+ achieve the highest QPS on both datasets for recall over 96.5%. This aligns with our earlier conclusion: SOTA methods often fail to maintain efficiency gains on OOD queries.

C. Extended Evaluation under Diverse Hardware Conditions

We further evaluate DCO methods across different hardware platforms on the XUltra dataset, which has the highest dimensionality among all evaluated datasets.

Performance on CPU with and without SIMD Support. As shown in Fig. 17a and Fig. 17b, PDScanning outperform than the SOTA methods due to the larger online pre-processing time for recall below 89.43 %. As the dimensionality further increases, DDCres attains the highest QPS, regardless of

whether SIMD is enabled. Specifically, it improves QPS by up to $1.7\times$ and $2.9\times$ over FDScanning on CPUs with and without SIMD, respectively.

Performance on GPU. As shown in Fig. 17c, DDCres consistently achieves the highest QPS. Notably, the bottleneck caused by online pre-processing time, observed on CPU, does not manifest on GPU. This can be attributed to the GPU's superior efficiency in matrix multiplication and parallel computation. The SOTA methods demonstrate strong performance on the ultra-high-dimensional XUltra dataset, improving QPS by up to $1.4\text{--}5.6\times$ over FDScanning.

D. Extended Evaluation on Pruning Capability of DCO

We evaluate and analyse the capability of DCO on IVF. Fig. 18 illustrates the dimension pruning ratios of DCOs and their recall across three datasets of varying dimensionality.

Pruning Capability of DCO on IVF. The results also demonstrate that the pruning effectiveness of DCOs is dimension-dependent: higher pruning ratios are consistently observed on high-dimensional and ultra-high-dimensional datasets. On these datasets, DDCopq typically achieves the highest pruning ratio. However, it is worth noting that, in addition to dimension scanning, DDCopq incurs additional overhead from computing quantized distances. Consequently, despite its strong pruning capability, DDCopq exhibits poorer query performance on IVF when SIMD is disabled. When SIMD is enabled, thereby significantly accelerating quantized distance computation on IVF, the performance of DDCopq aligns much more closely with its pruning capability. For other DCO methods, a higher pruning capability generally translates to better acceleration when SIMD is disabled. Moreover, all methods maintain comparable recall, demonstrating that the minor approximation errors introduced by DCOs do not compromise query accuracy on IVF.

Pruning Capability Comparisons: HNSW vs IVF. Combined with the findings in Fig. 6, we observe that the pruning capability of DCOs remains relatively stable on HNSW, whereas it increases with n_{probe} in IVF. This behavior can be attributed to the characteristics of candidate selection in each index structure. In HNSW, the retrieved candidates are typically close to the query vector. In contrast, IVF retrieves many candidates that are far from the query vector; for these, DCO methods can quickly determine irrelevance using only a few scanned dimensions. As a result, the pruning ratio in IVF rises with increasing n_{probe} .

E. Parameter Study

This experiment investigates two primary parameters of DCOs: the *initial step size* Δ_0 (*i.e.*, the initial number of dimensions to scan), and the *incremental step size* Δ_d used for each subsequent scan. We systematically evaluate their effects on both HNSW and IVF indexes using the GIST dataset. Our study primarily reports QPS, as recall remains largely unaffected. Notice that, prior work [28]–[30] usually sets these parameters as $\Delta_d = \Delta_0 = 32$.

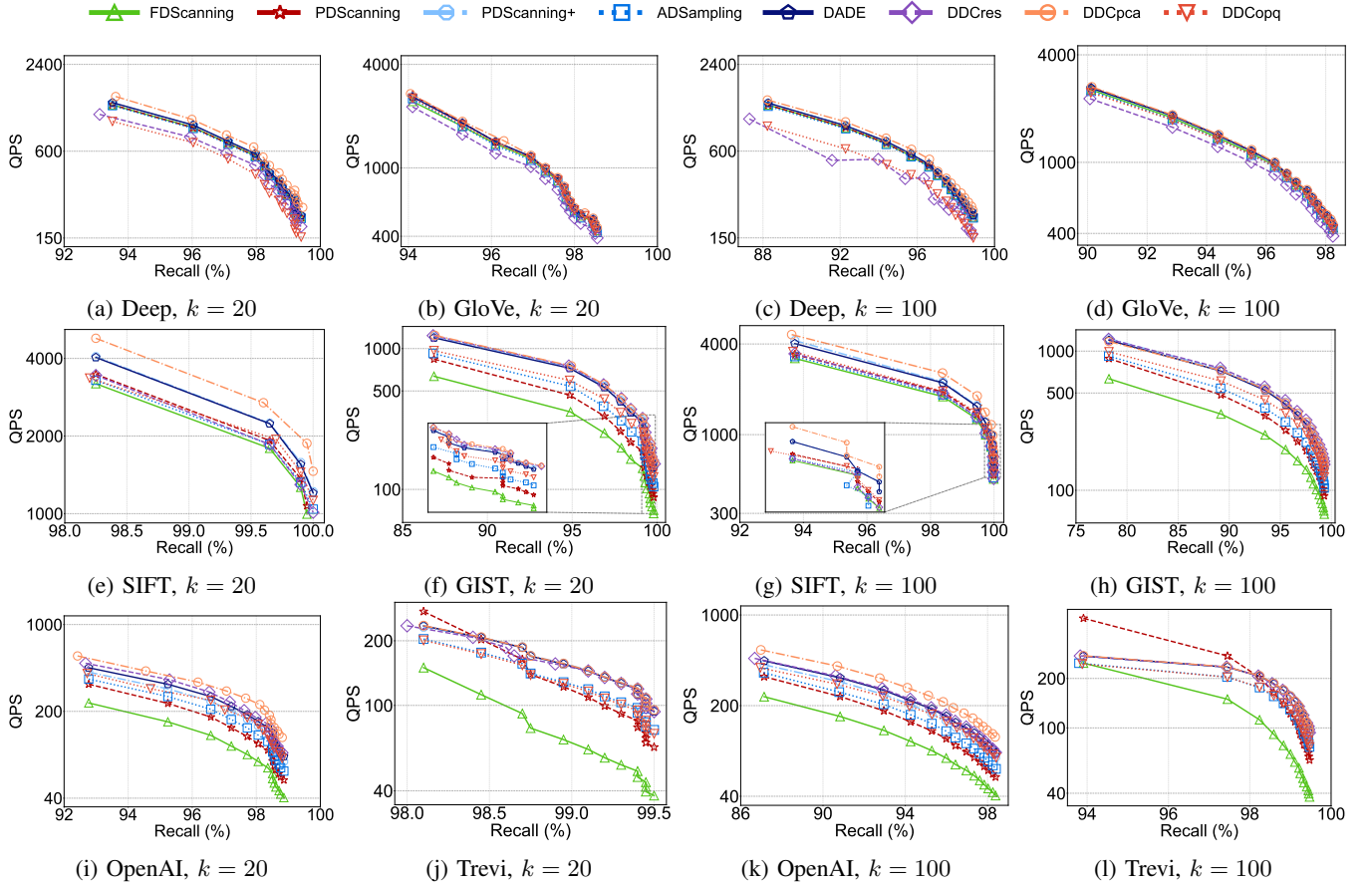


Fig. 15: Query performance when DCOs are applied to vector similarity search without SIMD

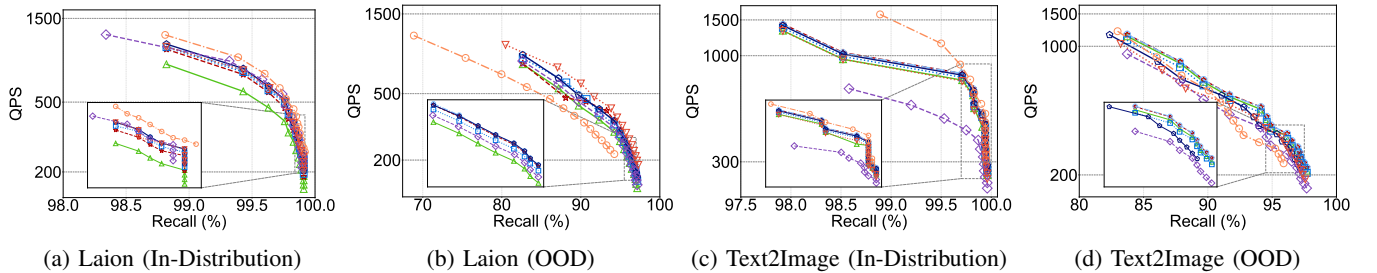


Fig. 16: Results on in-distribution and OOD queries without SIMD

Impact of Initial Step Size Δ_0 . Fig. 20 presents the QPS of each method on both HNSW and IVF indexes. In Fig. 19a, the QPS of simple scanning based methods **remains** relatively stable, whereas hypothesis testing based and classification based methods achieve a notable QPS increase by tuning Δ_0 appropriately. A similar trend is observed on the IVF index. ADSampling, DADE, DDCres, and DDCpca improve QPS by up to **1.3**, **1.5**, **1.2**, and **1.8** $\times$ respectively, in Fig. 19b.

The efficiency gain of tuning Δ_0 is more pronounced on IVF than on HNSW. This is because IVF stores each data partition contiguously in the main memory, a data layout that is **cache-friendly**. In contrast, the graph-based index HNSW lacks such a compact data layout. Consequently, even **PDScanning+**

achieves a considerable QPS improvement by adjusting Δ_0 in Fig. 19b.

Impact of Incremental Step Size Δ_d . Fig. 21 reports the QPS when varying Δ_d . Most algorithms are highly sensitive to this parameter, with FDScanning being the only exception. The optimal Δ_d is 64, 96 or 160 for HNSW, and 64 for IVF. Compared to the default $\Delta_d = 32$ in prior work [28]–[30], ADSampling, DADE, and DDCres achieve speedups of up to **1.1**, **1.2**, and **1.1** $\times$.

Overall, jointly using the optimal parameter combinations, ADSampling, DADE, and DDCres can boost QPS by up to **1.5**, **1.8**, and **1.8** $\times$ compared to the settings in prior work.

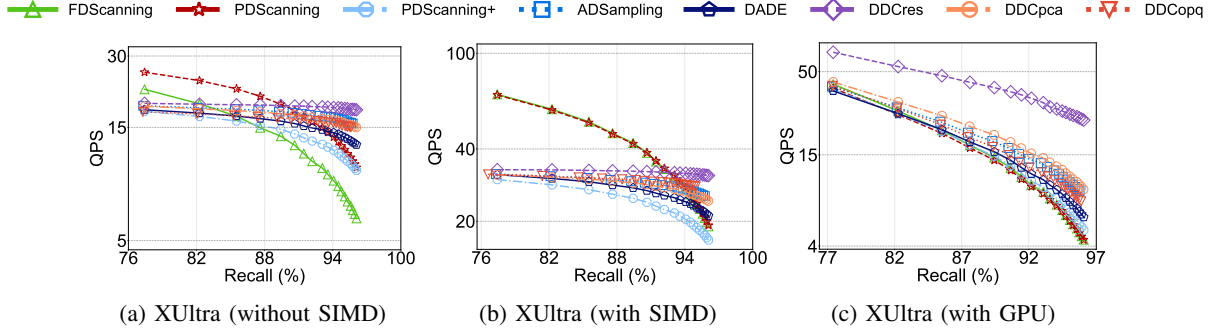


Fig. 17: Comparison of DCO methods on the IVF index across different hardware environments

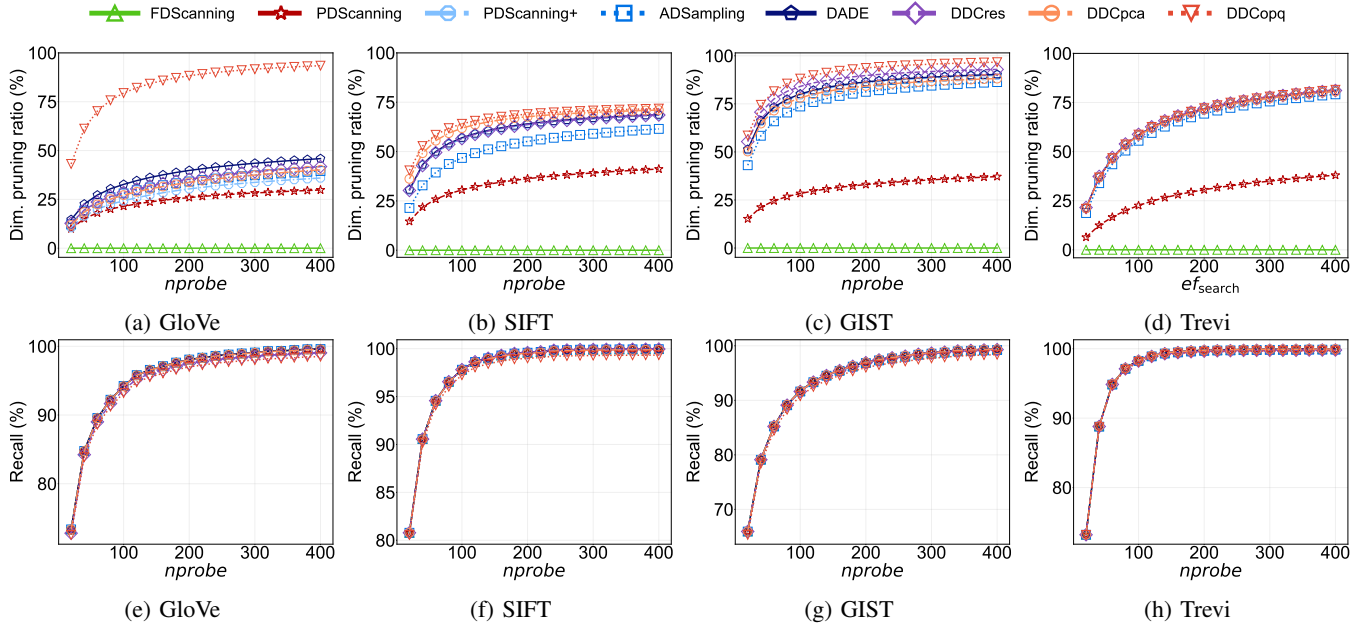


Fig. 18: Dimension pruning via DCO and its impact on recall on IVF

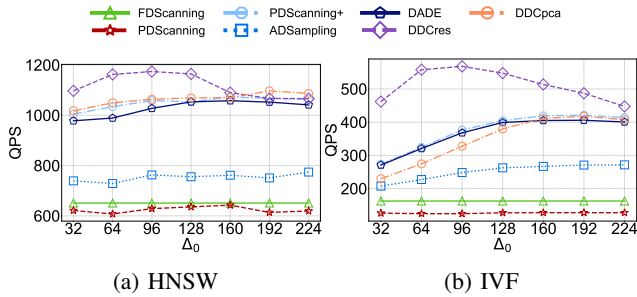


Fig. 20: Impact of initial step size Δ_0

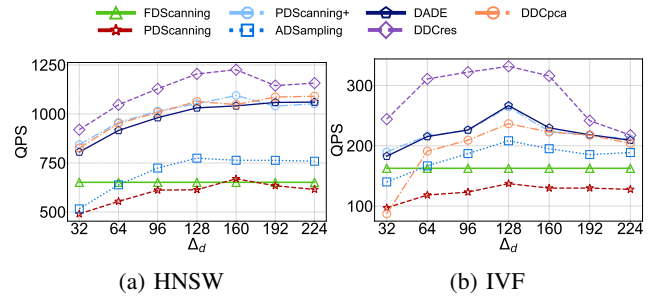


Fig. 21: Impact of incremental step size Δ_d